PPO and SAC: A Comparative Study

Sergio Herreros Pérez [1] **Adrián López-Lanchares Echezarreta** [1] **Ignacio Bayón Jiménez-Ugarte** [1]

Abstract

We study the application of Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC) to pixel-based continuous control tasks. While theoretically superior for continuous action spaces, PPO and SAC exhibit extreme hyperparameter instability when learning directly from high-dimensional visual observations. To address this, we leverage the Optuna framework to execute automated, short-horizon hyperparameter searches, rapidly identifying and pruning diverging configurations. We successfully scale these algorithms to full training horizons and compare their performance, sample efficiency, and stability against our prior Q-learning baselines.

1. Introduction

In our previous report, we demonstrated that value-based methods, such as Rainbow DQN, can successfully solve pixel-based continuous control tasks. However, adapting Q-learning to continuous domains requires action space discretization and even complex branching architectures to scale to more complex tasks. A more mathematically natural approach lies in policy gradient and actor-critic methodologies, which directly output continuous action distributions and are the standard choice for robotic locomotion.

In this study, we evaluate two state-of-the-art continuous control algorithms: Proximal Policy Optimization (PPO) (Schulman et al., 2017) and Soft Actor-Critic (SAC) (Haarnoja et al., 2018). While these algorithms represent the theoretical gold standard for physical control tasks, applying them directly to high-dimensional visual observations introduces deep practical challenges. Specifically, we found both algorithms to be highly sensitive to their underlying hyperparameter configurations. Even on the relatively simple Hopper environment, establishing a stable learning signal proved remarkably difficult. For instance, PPO exhibits extreme sensitivity to its target Kullback-Leibler (KL) divergence penalty; a threshold set too high causes the value loss to explosively diverge, while a threshold set too low results in premature policy collapse and a complete failure to learn.

To navigate this extreme hyperparameter sensitivity, we pivot our methodology from manual heuristic tuning to automated, large-scale search using the Optuna optimization framework (Akiba et al., 2019). Given the computationally expensive nature of pixel-based rendering, committing to long training horizons for unverified configurations is intractable. Instead, we designed an optimization pipeline that executes dozens of short-horizon trials. By evaluating the early learning dynamics over fewer environment steps, Optuna allows us to rapidly prune configurations that catastrophically diverge or stagnate. This broad but shallow search strategy effectively isolates the narrow, stable hyperparameter regimes required before scaling up to full training horizons.

2. Proximal Policy Optimization (PPO)

2.1. Algorithm Overview

Proximal Policy Optimization (PPO) (Schulman et al., 2017) is an on-policy policy gradient method that directly optimizes a stochastic policy $\pi_\theta(a|s)$ by maximizing the expected discounted return. To prevent destructively large policy updates, PPO introduces a clipped surrogate objective:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t \right) \right], \tag{1}$$

where $r_t(\theta) = {\pi_\theta(a_t|s_t)}/{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio between the current and old policy, $\hat{A}_t$ is an estimate of the advantage function, and ε is the clipping threshold. This formulation effectively constrains policy updates to a trust region without requiring computationally expensive second-order optimization.

PPO jointly optimizes a combined loss that also includes a value function term and an entropy bonus to encourage exploration:

$$\mathcal{L}(\theta) = \mathcal{L}^{\text{CLIP}}(\theta) - c_1\mathcal{L}^{\text{VF}}(\theta) + c_2\mathcal{H}[\pi_\theta], \tag{2}$$

[1]Master in Artificial Intelligence, Universidad Pontificia Comillas (ICAI). Correspondence to: Sergio Herreros Pérez <sergio.herreros@alu.comillas.edu>, Adrián López-Lanchares Echezarreta <adrianlanchares@alu.icai.comillas.edu>, Ignacio Bayón Jiménez-Ugarte <ibayon@alu.comillas.edu>.

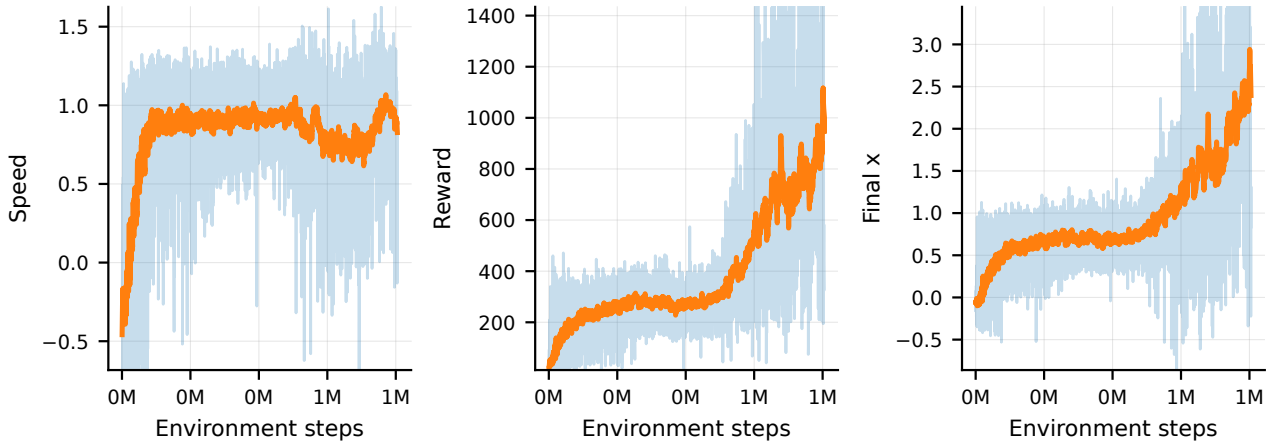

Figure 1. Training dynamics of PPO on the Hopper environment. (smoothed with an exponential moving average). [*TODO: Rehacer con steps K*]

where c_1 and c_2 are coefficients controlling the relative contribution of the value loss and the entropy bonus respectively. Advantages are estimated using Generalized Advantage Estimation (GAE) (Schulman et al., 2018), which balances bias and variance through a parameter λ:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t). \quad (3)$$

Unlike value-based algorithms like DQN, PPO operates natively in continuous action spaces by parameterizing the policy as a Gaussian distribution over actions. This makes it directly applicable to continuous control tasks without requiring any structural discretization or branching architectures.

2.2. Hopper: Learning from Pixels

We first validate PPO on the Hopper task, maintaining identical environment and reward configurations to our previous value-based experiments to ensure a fair comparison.

Configuration: The agent observes 64×64 pixel inputs with a frame stack of 4. We deploy 30 parallel environments to increase data throughput and stabilize advantage estimation, keeping the widened `healthy_angle_range` of $[-0.4, 0.4]$. The reward shaping remains focused on a target forward velocity of 1.5 units/s ($w_{\text{vel}} = 1.0$), with a smoothness penalty ($w_{\text{ctrl}} = 0.001$) and a survival bonus ($w_{\text{healthy}} = 1.0$). PPO is trained for 1.5 million environment frames. Rollouts collect 256 steps per parallel environment before executing 10 epochs of minibatch updates. These architectural and environment parameters were kept fixed throughout training. The remaining PPO-specific hyperparameters were selected via Optuna: a clipping threshold of $\varepsilon = 0.1$, a GAE parameter of $\lambda = 0.975$, and a discount factor of $\gamma = 0.995$. The policy and value networks share a convolutional encoder but maintain separate MLP heads, optimized via Adam with a learning rate of 4×10^{-4}.

Results and Discussion: PPO demonstrates vastly superior sample efficiency compared to both BDQN and Rainbow. In less than 1.5 million steps, the agent successfully learns to chain multiple forward hops without falling, whereas standard DQN required tens of millions of frames ($\sim 40M$) to reach a similar milestone. Rainbow achieves similar results in the same number of frames, but training is much slower.

However, the learned behavior is functionally sub-optimal. The agent's step-wise velocity converges to approximately 1.0 units/s, falling short of the 1.5 target. Rather than developing the rapid, fluid momentum observed in our best Rainbow models (which naturally exceeded the target), PPO discovers a more conservative strategy. It fumbles slightly between jumps to deliberately regain its balance before committing to the next hop. While the upward trajectory of the learning curve suggests further velocity improvements are possible with longer training horizons, we also noted severe hyperparameter brittleness. Specifically, the target KL-divergence threshold was exceptionally difficult to tune, often leading to premature policy collapse if not perfectly calibrated.

2.3. Humanoid: Scaling to High-Dimensional Control

Following the Hopper experiments, we scaled PPO to the Humanoid environment. PPO theoretically avoids the combinatorial explosion that crippled standard DQN by directly outputting a continuous Gaussian policy over all 17 joint torques.

Configuration: We applied the same morphological sim-

plifications as our prior experiments, freezing the Z-axis rotations of the abdomen and hips, disabling the arms entirely, and increasing the observation resolution to 96×96 pixels. To tackle the extreme hyperparameter sensitivity observed in the Hopper task, we utilized Optuna to conduct extensive automated hyperparameter searches over the PPO training parameters. We relied on hundreds of short-horizon trials to rapidly prune diverging configurations. For consistency with the Rainbow evaluation, we kept the environment constraints and reward weights fixed ($w_{\text{vel}} = 3.0$ and $w_{\text{healthy}} = 1.0$).

Results and Discussion: Despite PPO's native handling of continuous action spaces and the aggressive, automated search provided by Optuna, the agent failed to learn a stable bipedal gait. None of the attempted configurations managed to successfully balance and propel the Humanoid forward within the constrained trial bounds.

This failure largely stems from the fixed reward weights. While freezing the reward parameters was necessary to establish a direct comparison with Rainbow, it artificially restricted the optimization landscape for PPO. Actor-critic methods are notoriously sensitive to reward scaling and relative incentive ratios. Allowing Optuna to jointly tune the reward formulation alongside the core algorithm parameters, combined with a larger computational budget for extended trial horizons, would likely be required to successfully isolate a stable locomotion policy for this environment.

3. Soft Actor-Critic (SAC)

3.1. Algorithm Overview

Soft Actor-Critic (SAC) (Haarnoja et al., 2018) is an off-policy actor-critic algorithm that operates within the maximum entropy reinforcement learning framework. Unlike standard RL algorithms that solely aim to maximize the expected sum of rewards, SAC maximizes a dual objective consisting of both the expected return and the expected entropy of the policy:

$$J(\pi) = \sum_{t=0}^{T} \mathbb{E}_{(s_t,a_t)\sim\rho_\pi} \left[r(s_t, a_t) + \alpha \mathcal{H}\left(\pi(\cdot|s_t)\right) \right]. \quad (4)$$

Here, α is the temperature parameter that explicitly dictates the trade-off between exploitation (maximizing the reward) and exploration (maximizing the entropy $\mathcal{H}$).

To optimize this objective, SAC concurrently learns a stochastic policy $\pi_\theta(a|s)$ and two parameterized soft Q-functions, $Q_{\phi_1}(s, a)$ and $Q_{\phi_2}(s, a)$. The use of twin Q-networks mitigates the overestimation bias commonly found in value-based methods, taking the minimum of the two Q-values during policy evaluation. The policy itself is typically parameterized as a squashed Gaussian distribution, ensuring that sampled actions inherently respect the bounded continuous action space.

By aggressively reusing past experiences from a replay buffer, SAC theoretically bridges the gap between the high sample efficiency of off-policy methods (such as Rainbow) and the continuous control stability of on-policy methods (such as PPO). Furthermore, modern implementations of SAC automatically tune the temperature parameter α via dual optimization, ensuring that the exploration bounds dynamically adjust as the policy converges.

3.2. Hopper: Learning from Pixels

We evaluated SAC on the Hopper environment to directly compare its sample efficiency and computational demands against PPO and our value-based baselines.

Configuration: We maintain the foundational environment setup: 64×64 pixel inputs, a frame stack of 4, and the widened `healthy_angle_range` of $[-0.4, 0.4]$. Similar to our approach with PPO, we relied on Optuna to navigate the notoriously brittle hyperparameter landscape of continuous actor-critic methods. [*TODO: Separar y clarificar que parametros estaban fijos (e.g., replay buffer capacity, network architecture) y cuales fueron encontrados por Optuna (e.g., actor/critic learning rates, target entropy, batch size).*] The reward shaping remains fixed to the previous configuration to allow for a direct baseline comparison ($w_{\text{vel}} = 1.0$, target velocity 1.5, $w_{\text{ctrl}} = 0.001$, and $w_{\text{healthy}} = 1.0$).

Results and Discussion: In terms of pure environment interactions, SAC is by far the most sample-efficient algorithm evaluated in this study. The optimized configuration successfully learns a highly robust jumping gait, reaching approximately 5 times the cumulative reward and final forward distance in just [*TODO: fill exact number of steps, e.g., 500k*] frames. This drastically outpaces PPO (which required roughly 1.5M steps) and entirely eclipses standard DQN.

However, this exceptional sample efficiency comes at a severe computational cost. Because SAC performs computationally heavy gradient updates on three separate networks (the actor and two critics) at every environment step, its wall-clock training time is extremely slow. [*TODO: Specify how much time it takes more or less for SAC to train X steps compared to PPO.*] Additionally, much like PPO, SAC exhibited high instability during the tuning phase. It is particularly sensitive to the initial temperature parameter and the actor's learning rate, frequently resulting in catastrophic policy divergence if the entropy collapses too early.

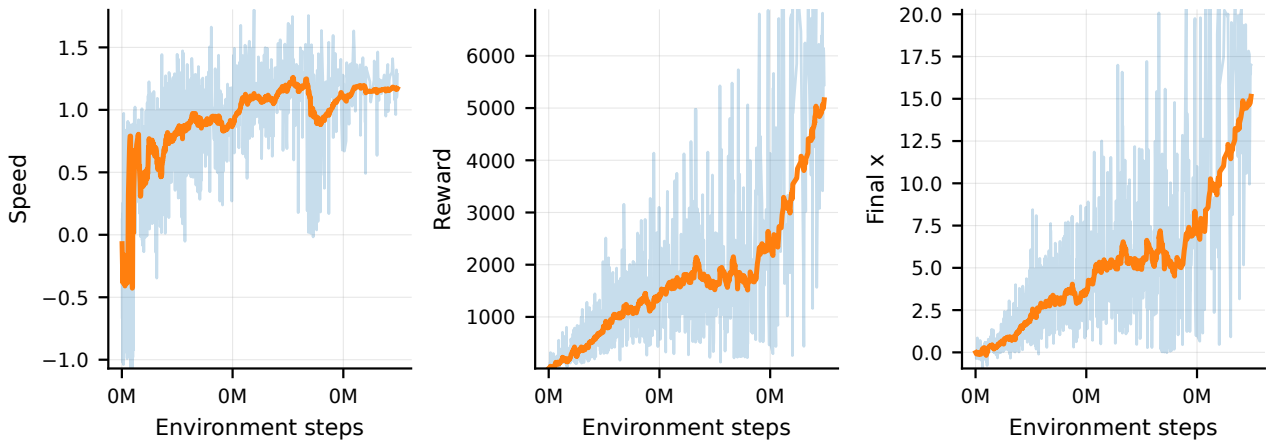

Figure 2. Training dynamics of SAC on the Hopper environment (smoothed with an exponential moving average) [*TODO: Rehacer con steps en miles (K)*]

3.3. Humanoid: Scaling to High-Dimensional Control

Given SAC's theoretical supremacy in sample-efficient continuous control, we attempted to scale it to the heavily constrained 96×96 pixel Humanoid environment.

Configuration: To give SAC the highest probability of success in this highly unstable bipedal domain, we relaxed the constraints on our Optuna search space. Recognizing that actor-critic methods are exceptionally sensitive to the scale and variance of the reward signal, we allowed Optuna to dynamically tune the reward weights (w_{vel}, w_{healthy}, and w_{ctrl}) alongside the core algorithmic parameters. [*TODO: Explain the few differentiating parameters tuned by Optuna here, such as the specific bounds for the reward weights, the polyak averaging coefficient (τ), or the delayed update frequencies.*]

Results and Discussion: The inclusion of reward shaping in the hyperparameter search yielded a highly promising configuration. During our short-horizon Optuna trials, we observed a specific run that successfully managed to balance the humanoid and take a couple of stable, coordinated steps forward. This indicates that SAC is fundamentally capable of extracting a viable bipedal locomotion policy directly from high-dimensional visual features.

Unfortunately, we were unable to train this promising configuration to full convergence. The sheer computational overhead of off-policy pixel-based SAC on a 17-joint continuous action space is paralyzing. The time required to process the convolutional feature extraction, update the twin critics, and optimize the Gaussian actor at every single step made extended training horizons practically unfeasible within our compute budget. Consequently, while SAC demonstrates the highest theoretical ceiling for pixel-based continuous control, its deployment is severely bottlenecked by its immense wall-clock training time.

4. PPO vs. SAC: A Comparative Analysis

While both Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC) operate natively in continuous action spaces, our experiments reveal stark trade-offs between sample efficiency, computational overhead, and hyperparameter stability.

Sample Efficiency and Performance: SAC overwhelmingly dominates in terms of pure sample efficiency. By aggressively reusing off-policy data from a replay buffer and leveraging a maximum entropy objective, SAC learned a highly robust and optimal Hopper gait in merely a fraction of the environment steps required by PPO (approximately 500k frames compared to 1.5M). Furthermore, SAC achieved significantly higher cumulative rewards, whereas PPO converged to a sub-optimal, conservative hopping strategy. On the complex Humanoid environment, SAC was the only actor-critic method that demonstrated the capacity to learn coordinated bipedal steps, whereas PPO completely failed to find a viable policy under the same constrained trial horizons.

Computational Cost: Conversely, PPO drastically outperforms SAC in wall-clock training time. PPO performs policy updates iteratively on batched rollouts, making its per-step computational footprint relatively lightweight. SAC, however, updates an actor network and two separate critic networks at every single environment step. When processing high-dimensional visual features through convolutional encoders, this continuous gradient calculation becomes a severe computational bottleneck, making extended training horizons painfully slow.

Hyperparameter Brittleness: Both algorithms share a crit-

ical weakness: extreme hyperparameter sensitivity. Unlike standard Q-learning, which tends to fail gracefully, actor-critic methods in visual domains are highly prone to catastrophic divergence. PPO is exceptionally sensitive to its target KL-divergence and clipping thresholds, while SAC requires delicate tuning of its temperature parameter and learning rates to prevent premature entropy collapse. Both necessitated aggressive, automated search strategies via Optuna to isolate stable learning regimes.

5. Conclusion

This study along with the previous one systematically evaluated the application of four distinct Deep Reinforcement Learning algorithms (Branching DQN, Rainbow, PPO, and SAC) to the challenging domain of pixel-based continuous control. By benchmarking these methods across the DeepMind Control Suite, we identified the fundamental limitations and practical trade-offs inherent to both value-based and policy gradient architectures.

Our findings demonstrate that classic value-based methods can indeed be adapted for continuous control. While standard Branching DQN scales poorly to complex morphologies and completely failed on the bipedal Humanoid task, heavily augmented architectures like Rainbow DQN proved surprisingly capable. Through the integration of distributional value modeling, multi-step returns, and carefully calibrated reward shaping, Rainbow successfully learned stable, albeit visually unnatural, bipedal locomotion directly from raw pixels.

However, value-based methods require cumbersome action-space discretization. Actor-critic algorithms like PPO and SAC provide a more mathematically elegant solution by directly outputting continuous action distributions. Unfortunately, our experiments reveal that applying these algorithms to high-dimensional visual inputs introduces profound hyperparameter brittleness. While PPO is computationally fast per step, it suffers from poor sample efficiency and policy collapse if not perfectly tuned. SAC represents the theoretical ceiling for sample efficiency and performance, successfully taking bipedal steps where PPO failed, but its deployment is severely bottlenecked by paralyzing wall-clock training times and massive computational overhead.

Ultimately, achieving stable continuous control from pixels requires navigating a strict trade-off matrix. Rainbow DQN offers a slow but resilient path through complex value estimation, whereas SAC offers explosive sample efficiency gated by heavy compute requirements. Future research must focus on bridging this gap, potentially by exploring hybrid architectures that combine the stability of distributional Q-learning with the continuous native outputs of maximum-entropy actor-critic frameworks.

References

Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pp. 2623–2631, 2019.

Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pp. 1861–1870. PMLR, 2018.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms, 2017. URL https://arxiv.org/abs/1707.06347.

Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. High-dimensional continuous control using generalized advantage estimation, 2018. URL https://arxiv.org/abs/1506.02438.